多域名 Nginx 迁移方案

> 目标：将静态网站从 backend 镜像中剥离，由 Nginx 统一对外托管，backend 仅暴露内网 API。

架构对比

迁移前

```
Internet :80/:443
    │
 [backend 容器]  ← 同时托管 API + 4 个静态站
    ├── shelf_static (sango/)
    ├── shelf_static (sangoplay/)
    ├── shelf_static (mini/)
    ├── shelf_static (public/)
    └── Dart API 路由
```

迁移后

```
Internet :80/:443
    │
 [Nginx 容器]
    ├── sangogame.com          → /var/www/sangogame/
    │        └── /sangoplay/   → 301 重定向到 play.sangogame.com
    ├── play.sangogame.com     → /var/www/sangoplay/
    ├── ibookjoy.com           → /var/www/ibookjoy/
    └── ycwithyou.com          → /var/www/ycwithyou/
            ├── /api/*         → backend:8080 (内网)
            └── /ws/chat/*     → backend:8080 (内网, WebSocket)

 [backend 容器] :8080 — 仅内网，不对外绑定端口
 [certbot 容器]         — Let's Encrypt 证书自动申请/续签
```

域名映射表

域名	静态内容来源	服务器目录	API 代理
`sangogame.com`	`sangogame.com/out/`	`/opt/iwithyou/static/sangogame/`	无
`play.sangogame.com`	`backend/sangoplay/`（Flutter build）	`/opt/iwithyou/static/sangoplay/`	无
`ibookjoy.com`	`ibookjoy.com/out/`	`/opt/iwithyou/static/ibookjoy/`	无
`ycwithyou.com`	`ycwithyou.com/out/`	`/opt/iwithyou/static/ycwithyou/`	`/api/*` 和 `/ws/chat/*` → backend:8080

文件索引

迁移涉及的所有新增/修改文件：

文件	类型	说明
`backend/deploy/docker-compose.yml`	新增	服务编排，上传到服务器
`backend/deploy/nginx/conf.d/sangogame.conf`	新增	sangogame.com Nginx 配置
`backend/deploy/nginx/conf.d/sangoplay.conf`	新增	play.sangogame.com Nginx 配置
`backend/deploy/nginx/conf.d/ibookjoy.conf`	新增	ibookjoy.com Nginx 配置
`backend/deploy/nginx/conf.d/ycwithyou.conf`	新增	ycwithyou.com Nginx 配置
`backend/deploy/run_compose.sh`	新增	替换服务器上的 run_New.sh
`sangogame.com/deploy.sh`	新增	sangogame.com 独立部署
`ibookjoy.com/deploy.sh`	新增	ibookjoy.com 独立部署
`ycwithyou.com/deploy.sh`	新增	ycwithyou.com 独立部署
`backend/deploy_sangoplay.sh`	新增	sangoplay 独立部署
`ibookjoy.com/next.config.ts`	修改	删除 basePath（根路径部署）
`backend/Dockerfile`	修改	删除 4 个静态文件 COPY

`backend/bin/server.dart`	修改	删除 shelf_static 路由挂载
`frontend/scripts/version_config.conf`	修改	服务器 URL 改为 https://
`frontend/scripts/upload_version.sh`	修改	默认服务器 URL 改为 https://

服务器环境要求

在开始任何服务器操作前，按以下清单逐项检查，全部通过再继续。

📋 本地机器检查（执行部署脚本的机器）

L1. 设置 SERVER_IP 环境变量（所有脚本的前提）

```
export SERVER_IP=<服务器公网IP>
echo $SERVER_IP    # 确认非空
```

L2. SSH 免密登录已配置

```
ssh root@$SERVER_IP "echo ok"
# 期望：直接返回 ok，无密码提示
```

如需配置：

```
ssh-copy-id root@$SERVER_IP
```

L3. rsync 已安装（deploy.sh 依赖）

```
rsync --version
# 期望：rsync version 3.x.x
```

macOS 默认自带，如缺失：`brew install rsync`

📋 服务器端检查（SSH 登录后执行）

S1. Docker Engine 已安装且运行（必须 ≥ 20.10）

```
systemctl status docker | grep Active
docker --version
# 期望：Active: active (running)，Docker version 20.10+
```

如未安装：https://docs.docker.com/engine/install/

S2. Docker Compose Plugin v2（必须，使用 `docker compose` 而非 `docker-compose`）

```
docker compose version
# 期望：Docker Compose version v2.x.x
```

> ⚠️ 本方案所有命令均使用 `docker compose` （v2 plugin），不兼容旧版 `docker-compose` （v1）。
>
> 如返回 "command not found"，安装方式：
>
> ```bash
> # Ubuntu/Debian
> apt-get install docker-compose-plugin
> # CentOS/RHEL
> yum install docker-compose-plugin
> ```

S3. 端口 80 / 443 未被占用，且宿主机无系统级 Nginx

```
ss -tlnp | grep -E ':80\b|:443\b'
# 期望：无任何输出（表示端口空闲）
```

如有占用，先确认是否为宿主机 Nginx（会与 Docker nginx 容器冲突）：

```
systemctl status nginx 2>/dev/null || echo "no system nginx"

# 如有，停止并禁用开机自启
systemctl stop nginx && systemctl disable nginx
```

其他占用进程（如旧容器）在 Phase 3 停止旧容器后会自动释放。

S4. 云安全组已开放 80 / 443 端口

> ⚠️ 这是最常见的踩坑点。 `ss` 只看 OS 层端口，云厂商安全组是外层防火墙，两者独立。
>
> 腾讯云操作路径：轻量服务器控制台 → 防火墙 → 添加规则：
>
> - 协议 TCP，端口 80，来源 0.0.0.0/0
>
> - 协议 TCP，端口 443，来源 0.0.0.0/0

验证安全组是否生效（在本地机器执行）：

```
nc -zv $SERVER_IP 80
nc -zv $SERVER_IP 443
# 期望：Connection to ... succeeded（或 refused 但非 timeout，timeout = 安全组未放行）
```

S5. 系统防火墙未拦截 80 / 443

```
# 检查 ufw
ufw status 2>/dev/null | grep -E '80|443|Status'

# 检查 firewalld
```

```
firewall-cmd --list-ports 2>/dev/null

# 如有拦截，放行：
ufw allow 80/tcp && ufw allow 443/tcp
```

S6. 系统时间同步正常（Let's Encrypt 要求误差 < 2 分钟）

```
timedatectl status | grep -E 'Local time|synchronized'
# 期望：System clock synchronized: yes
```

如未同步：

```
# Ubuntu/Debian
apt-get install -y ntp && systemctl enable --now ntp

# 或手动同步
ntpdate -u pool.ntp.org
```

S7. 服务器可访问公网（拉取 Docker 镜像需要）

```
curl -I https://registry-1.docker.io --max-time 10
# 期望：HTTP/2 200 或 301（能连通即可）
```

如有限速或超时，可配置国内镜像加速器（腾讯云服务器推荐使用自带的镜像源）。

S8. Tencent CCR 私有仓库登录（backend 镜像来源）

```
docker login ccr.ccs.tencentyun.com
# 输入腾讯云账号的访问密钥（非控制台密码）
```

验证可拉取镜像：

```
docker pull ccr.ccs.tencentyun.com/ephnic/newserver:latest
# 期望：Pull complete，无 authentication required 报错
```

S9. 磁盘空间（建议 /opt 下 ≥ 5GB 可用）

```
df -h /opt
# 期望：Avail ≥ 5G
```

S10. 内存（建议 ≥ 1GB 可用）

```
free -h
# 期望：available ≥ 1G
```

S11. 记录当前运行容器（操作前留档，用于回滚）

```
docker ps --format "table {{.ID}}\t{{.Image}}\t{{.Ports}}\t{{.Names}}"
```

> 记录旧容器 ID，Phase 3 停止前不要删除。

实施步骤

Phase 1 — 本地代码修改（已完成，可直接验证）

1.1 ibookjoy.com — 去掉 basePath

文件： `ibookjoy.com/next.config.ts`

```
- const BASE_PATH = '/mini';
  const nextConfig: NextConfig = {
    output: 'export',
-   basePath: BASE_PATH,
-   env: {
-     NEXT_PUBLIC_BASE_PATH: BASE_PATH,
-   },
    images: {
      unoptimized: true,
    },
  };
```

验证： `npm run build` ，检查 `out/index.html` 中资源路径是否为 `/_next/` 而非 `/mini/_next/`

1.2 backend — Dockerfile 精简

文件： `backend/Dockerfile` ，删除末尾 4 行静态 COPY：

```
- COPY --from=build /app/public    /app/public
- COPY --from=build /app/sango     /app/sango
- COPY --from=build /app/sangoplay /app/sangoplay
- COPY --from=build /app/mini      /app/mini
```

1.3 backend — server.dart 精简

文件： `backend/bin/server.dart` ，删除 sango / sangoplay / mini 静态路由挂载（约 7 行），以及 `Cascade` 里的 `createStaticHandler('public', ...)` 行。

Phase 2 — DNS 解析配置 ✅ 已完成

三个域名及 sangoplay 子域名已在腾讯云 DNS 解析控制台完成 A 记录配置，均解析到服务器公网 IP。

> `play.sangogame.com` 需单独添加一条 A 记录：主机记录 `play` ，记录值填服务器公网 IP。

SSL 证书已通过腾讯云 SSL 控制台申请完成（见 Phase 5）。

验证命令（确认解析生效）：

```
nslookup sangogame.com
nslookup play.sangogame.com
nslookup ibookjoy.com
nslookup ycwithyou.com
# 四个域名均应返回服务器公网 IP
```

Phase 3 — 服务器初始化（SSH 登录执行）

> 前提：Phase 2 DNS 解析已生效，`nslookup 域名` 能正确返回服务器 IP。

```
# 登录服务器
ssh root@$SERVER_IP

# 创建目录结构
mkdir -p /opt/iwithyou/nginx/conf.d
mkdir -p /opt/iwithyou/certs/
{sangogame.com,play.sangogame.com,ibookjoy.com,ycwithyou.com}
mkdir -p /opt/iwithyou/static/{sangogame,sangoplay,ibookjoy,ycwithyou}
mkdir -p /opt/iwithyou/uploads

# 停止旧容器（先记录容器 ID）
docker ps
docker stop <old_container_id>
# 先不 rm，等新架构验证通过后再清理
```

Phase 4 — 上传配置文件（本地执行）

```
cd /Users/wanxt/app/iwithyou/backend/deploy

# 上传 docker-compose 和启动脚本
scp docker-compose.yml root@$SERVER_IP:/opt/iwithyou/
scp run_compose.sh root@$SERVER_IP:/opt/iwithyou/

# 上传 Nginx 配置（已是正式 HTTPS 配置，可直接上传）
scp -r nginx/conf.d root@$SERVER_IP:/opt/iwithyou/nginx/

# 上传服务器端 .env（从 .env_remote 复制，敏感信息不要提交 git）
scp ../.env_remote root@$SERVER_IP:/opt/iwithyou/.env
```

Phase 5 — 上传腾讯云 SSL 证书

> SSL 证书已在腾讯云控制台申请完成，此步骤将本地证书文件上传到服务器并挂载到 nginx 容器。

5.1 从腾讯云控制台下载证书

腾讯云控制台 → SSL 证书 → 证书列表 → 下载，选择 **Nginx** 格式。

下载后解压，每个域名得到一个目录，内含两个文件：

```
域名_nginx/
    └── 域名_bundle.crt    ← 证书链（包含中间证书）
```

```
    └── 域名.key              ← 私鉅
```

5.2 将证书放到规范目录

在本地整理指定目录结构（与 docker-compose.yml 挂载路径对应）：

```
# 本地在 backend/deploy/ 下创建 certs 目录
mkdir -p /Users/wanxt/app/iwithyou/backend/deploy/certs/sangogame.com
mkdir -p /Users/wanxt/app/iwithyou/backend/deploy/certs/play.sangogame.com
mkdir -p /Users/wanxt/app/iwithyou/backend/deploy/certs/ibookjoy.com
mkdir -p /Users/wanxt/app/iwithyou/backend/deploy/certs/ycwithyou.com

# 将下载的证书文件复制到对应目录
# 以 sangogame.com 为例:
cp 下载路径/sangogame.com_nginx/sangogame.com_bundle.crt \
   certs/sangogame.com/
cp 下载路径/sangogame.com_nginx/sangogame.com.key \
   certs/sangogame.com/

# play.sangogame.com（单独申请的免费证书）:
cp 下载路径/play.sangogame.com_nginx/play.sangogame.com_bundle.crt \
   certs/play.sangogame.com/
cp 下载路径/play.sangogame.com_nginx/play.sangogame.com.key \
   certs/play.sangogame.com/

# ibookjoy.com 和 ycwithyou.com 同理
```

> ⚠️ `certs/` 目录包含私鉅，已在 `.gitignore` 中排除，不要提交到 git。

5.3 上传证书到服务器

```
cd /Users/wanxt/app/iwithyou/backend/deploy

# 在服务器创建 certs 目录
ssh root@$SERVER_IP "mkdir -p /opt/iwithyou/certs"

# 上传全部证书
scp -r certs/ root@$SERVER_IP:/opt/iwithyou/
```

5.4 证书有效期管理

腾讯云 SSL 证书到期后需要手动或自动续费。副作用：证书延期即中断 HTTPS。

建议在证书到期前 30 天更新证书文件：

1. 腾讯云控制台续费/重新申请 → 下载新证书
2. 替换本地 `certs/` 下对应文件
3. 重新执行 5.3 上传命令
4. 服务器执行：

```bash
docker compose exec nginx nginx -s reload
```

```

---

## Phase 6 — 首次部署静态文件（本地执行）

```
sangogame.com
cd /Users/wanxt/app/iwithyou/sangogame.com
bash deploy.sh

ibookjoy.com
cd /Users/wanxt/app/iwithyou/ibookjoy.com
bash deploy.sh

ycwithyou.com
cd /Users/wanxt/app/iwithyou/ycwithyou.com
bash deploy.sh

sangoplay（Flutter web，源文件在 backend/sangoplay/）
cd /Users/wanxt/app/iwithyou/backend
bash deploy_sangoplay.sh
```

---

## Phase 7 — 启动完整服务（服务器执行）

```
cd /opt/iwithyou

# 启动所有服务
docker compose up -d

# 验证
docker compose ps
curl -I https://sangogame.com
curl -I https://play.sangogame.com
curl -I https://ibookjoy.com
curl -I https://ycwithyou.com
curl https://ycwithyou.com/api/version/latest
curl https://play.sangogame.com/
```

Phase 8 — 更新 release.command（后端发布流程）

`release.command` 末尾 SSH 调用改为：

```
REMOTE_SCRIPT="bash /opt/iwithyou/run_compose.sh $REMOTE_IMAGE_NAME $
{new_version}"
sshpass -p "$SERVER_PASSWORD" ssh root@$SERVER_IP "$REMOTE_SCRIPT"
```

```

Phase 9 — 清理旧容器

验证所有站点正常后：

```
ssh root@$SERVER_IP
docker rm <old_container_id>    # 清理旧 backend 容器
docker rmi <old_image_id>       # 清理旧镜像（可选）
```

验证清单

检查项	命令	期望结果
HTTP → HTTPS 重定向	`curl -I http://sangogame.com`	`301 → https://`
HTTPS 证书有效	`curl -I https://sangogame.com`	`200 OK`
sangogame 主页	`curl https://sangogame.com/`	Next.js 静态 HTML
sangoplay 主页	`curl https://play.sangogame.com/`	Flutter web index
sangogame.com/sangoplay 重定向	`curl -I https://sangogame.com/sangoplay/`	`301 → https://play.sangogame.com/`
ibookjoy 主页	`curl https://ibookjoy.com/`	Next.js 静态 HTML
ycwithyou 主页	`curl https://ycwithyou.com/`	Next.js 静态 HTML
API 代理	`curl https://ycwithyou.com/api/version/latest`	`{"version":"..."}`
WebSocket 握手	见下方命令	`101 Switching Protocols`
backend 不对外暴露	`curl http://<SERVER_IP>:8080`	连接拒绝

WebSocket 握手验证命令：

```
curl --http1.1 -i \
  -H "Upgrade: websocket" \
  -H "Connection: Upgrade" \
  -H "Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==" \
  -H "Sec-WebSocket-Version: 13" \
  "https://ycwithyou.com/ws/chat/test?name=probe"
# 期望返回 101 Switching Protocols
# 注意：必须强制 HTTP/1.1，HTTP/2 不支持 Upgrade 握手
```

回滚方案

如新架构出现问题，可立刻回到旧容器：

```
ssh root@$SERVER_IP
docker compose down
docker run ... <old_image>   # 按旧 run_New.sh 参数启动
```

由于旧容器没有删除，回滚仅需 30 秒。